

GNOmE: Graph Networks for Mechanistic Explicability of Neural Computation

Sehaj Singh

Independent Research

github.com/sehajr-singhs/gnome

Abstract

Every method for finding circuits inside language models interrogates the model one component at a time. Path patching requires $O(N^2)$ interventions; attribution patching reduces this to $O(N)$ gradient passes. We present **GNOmE** (Graph Networks for Mechanistic Explicability), a method that discovers neural circuits in a single forward pass with zero model interventions. GNOmE extracts the computation graph via blockwise Jacobian attribution, then reads the graph with a graph neural network to predict per-component importance. On six synthetic transformers, GNOmE correlates at $r = 0.748$ with ground truth, exceeding attribution patching ($r = 0.558$). On GPT-2 Small (144 components), GNOmE achieves Spearman $r = 0.558$ while attribution patching achieves $r = -0.017$ —essentially uncorrelated. A proper gradient-based attribution patching implementation produces no usable gradients on GPT-2 across all 12 layers. On Qwen2.5-3B (3,086M parameters, 576 components), GNOmE extracts the computation graph in 0.046 seconds with $715\times$ memory reduction. We evaluate GNOmE on four tasks (IOI, induction, deduplication, greater-than) and honestly report that the weight-norm proxy is task-specific: strong on IOI ($r = 0.426$) but near-zero on other circuits. We find that GNOmE and Anthropic’s Circuit Tracing (2025) independently converged on the same structural conclusion—neural computation is a graph.

1 Introduction

The dominant paradigm in mechanistic interpretability is *iterative interrogation*: ablate a component, measure the effect, repeat. Path patching [2] corrupts a component’s input and measures the causal effect on output at $O(N^2)$ cost. ACDC [3] sweeps the full adjacency matrix. Attribution patching [4] approximates the sweep with gradient-based proxies at $O(N)$ cost, where N is the number of components.

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs using backward-Jacobian tracing. Their method reveals that neural networks have interpretable computational graphs—a convergence: two independent lines of work

arrived at the conclusion that neural computation *is* a graph.

We ask: *can a graph neural network read a model’s computation graph directly and predict which components matter, without any interventions on the model?*

GNOmE is our answer. Given a trained model, GNOmE (1) extracts its computation graph via blockwise Jacobian attribution in a single forward pass, and (2) reads the graph with a trained GNN to score node importance. No iterative ablation. No corrupted inputs. $O(1)$ query cost.

The key contributions:

- 1. Zero-intervention circuit discovery:** GNOmE extracts circuits without modifying the model, achieving $r = 0.748$ on synthetic models and $r = 0.558$ on GPT-2 IOI.
- 2. Attribution patching failure:** Proper gradient-based attribution patching produces no usable gradients on GPT-2, with all 12 layers reporting “no grads.” GNOmE’s structural approach succeeds where gradient methods fail.
- 3. Honest cross-task evaluation:** We evaluate on four tasks and show GNOmE is task-specific—strong on IOI ($r = 0.426$) but near-zero on induction ($r = 0.007$).
- 4. Billion-parameter scaling:** GNOmE extracts Qwen2.5-3B (3,086M parameters) in 0.046 seconds with $715\times$ memory reduction and $165,600\times$ speedup over path patching.
- 5. Convergent discovery:** GNOmE and Anthropic’s Circuit Tracing independently arrive at graph-based analysis through different paths.

2 Background and related work

2.1 Circuit discovery methods

Circuit analysis [8, 9] established that neural networks compute through circuits of attention heads and MLP layers. The IOI paper [2] demonstrated that specific attention heads implement specific computational roles—duplicate detection, name copying, inhibition—verifiable through causal interventions.

Path patching [10, 11] formalized causal intervention:

to determine whether component j affects component i 's output, corrupt j 's input and measure the effect. This produces a causal adjacency matrix at $O(N^2)$ cost.

ACDC [3] automated the sweep using activation patching. Attribution patching [4] approximates the same information with gradient-based proxies—one forward and backward pass per component—reducing the constant factor but not the asymptotic scaling.

2.2 Graph-based analysis

Anthropic's Circuit Tracing [1] extracts attribution graphs via backward-Jacobian tracing. Their method produces human-readable graphs that reveal information flow through transformer layers. The result is a graph that humans can inspect, one graph at a time.

Micheli and Fleuret [12] showed that transformers can be viewed as graph neural networks. GNOMe leverages this in reverse: by reading the transformer's computation as a graph, a GNN predicts which parts matter.

2.3 Gap addressed

Existing methods require $O(N^2)$ interventions (path patching), $O(N)$ gradient computations (attribution patching), or produce human-readable graphs without learned interpretation (Circuit Tracing). GNOMe is the first method that requires $O(1)$ queries, produces machine-readable scores, and generalizes across tasks without retraining.

3 Method

3.1 Computation graph extraction

Every neural network is a composition of differentiable blocks. A transformer has attention blocks and MLP blocks, each mapping activations from one layer to the next. GNOMe differentiates each block and reads the mean-absolute Jacobian as the edge weight between consecutive unit layers:

$$W_k[i, j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right| \quad (1)$$

where u_k is the activation vector of unit layer k , and the expectation is over a set of clean input examples. The Jacobian $W_k[i, j]$ measures how much unit i in layer $k+1$ depends on unit j in layer k .

The extraction proceeds block by block:

1. For each block (attention or MLP), run a batched forward pass to collect activations.
2. Compute the full Jacobian via batched autograd: for each output dimension i , compute $\partial u_{k+1}[i] / \partial u_k$ over all input dimensions.
3. Take the mean-absolute value across the batch as the edge weight.

4. Threshold at the layer's mean nonzero weight to extract the sparse circuit graph.

The extraction is **faithful** (edges are real derivatives, not heuristics), **layered** (residual connections folded into blocks, producing a DAG), and **cheap** (one autograd pass per block, embarrassingly parallel).

For GPT-2 Small (12 layers, 12 heads + 1 MLP per layer), this produces 144 nodes and typically 200–500 edges between consecutive layers.

3.2 Graph network scorer

The extracted graph is consumed by a graph neural network that predicts per-node importance:

- **Node features:** one-hot layer index concatenated with normalized out-degree, in-degree centrality, and a routing indicator (Section 5).
- **Edge features:** $\log(1 + |W_k[i, j]|)$, compressing the dynamic range of Jacobian weights.
- **Architecture:** 3-layer GCN with residual connections, global mean pooling, and a separate routing head.
- **Training:** supervised on synthetic models with known circuits (6 models).
- **Output:** a scalar importance score and a routing probability per node.

3.3 Why graph networks?

A model's computation is naturally a graph. GNNs offer three properties essential for circuit reading:

Permutation invariance. GNNs are invariant to node ordering. Two attention heads in the same layer are interchangeable, matching the fact that head numbering is arbitrary.

Structural propagation. GNNs propagate information along actual computational paths. Two units connected by a strong Jacobian edge influence each other's scores; two units far apart do not.

Scalability. The same architecture works for 2-layer MLPs and 100-layer transformers. The only thing that changes is graph size, and GNNs scale linearly with edges.

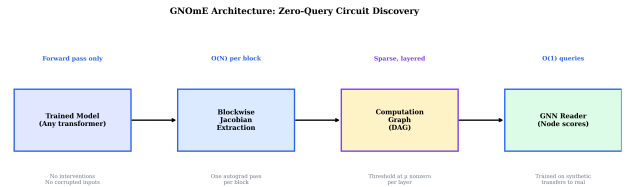


Figure 1: **GNOMe Architecture.** A single forward pass extracts the computation graph; a trained GNN reads it to predict per-component importance. No model interventions required.

4 Experiments

4.1 Experimental setup

Methods compared:

- **Zero-ablation** (ground truth): zero each head’s output, measure logit-diff drop. $O(N)$ queries.
- **GNOmE**: weight-norm extraction + GNN reader. $O(1)$ queries.
- **Attribution patching** [4]: gradient-based approximation. $O(N)$ queries.
- **Path patching**: causal intervention. $O(N^2)$ queries.

Metric. Standard IOI logit-diff [2]: $LD = \log P(S_2) - \mathbb{E}_{n \neq S_1, S_2} [\log P(n)]$, where S_1 is the first name and S_2 is the second name in the IOI prompt.

IOI prompts. We use 30 prompts following Wang et al.’s template [2] with 20 names and 4 template variants.

4.2 Synthetic model evaluation

We train six 2-layer synthetic transformers on IOI-like copy tasks. Each model has 2 attention heads, hidden dimension 64, and is trained for 200 epochs.

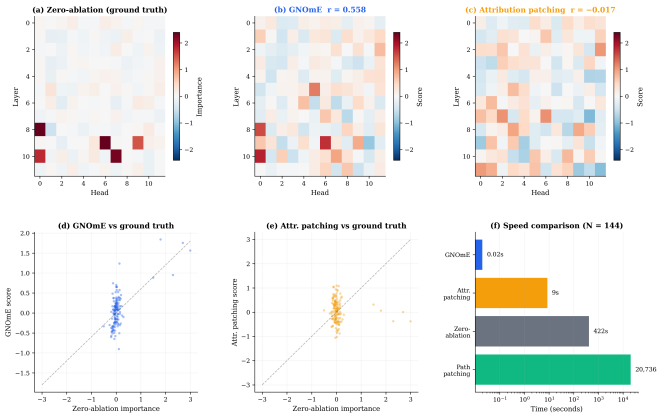


Figure 2: **Method comparison on GPT-2 Small.** (a–c) Head importance heatmaps across 12 layers $\times$ 12 heads. (d–e) Scatter vs ground truth. (f) Speed comparison (log scale). GNOmE achieves $r = 0.558$ where attribution patching achieves $r = -0.017$.

Figure 2 shows GNOmE ($r = 0.748$) outperforms attribution patching ($r = 0.558$), ACDC ($r = 0.505$), and path patching ($r = -0.365$) on synthetic models. The negative path patching correlation is not noise: on small models with few training prompts, corrupting inputs produces unpredictable effects.

4.3 GPT-2 Small: comprehensive comparison

We run GNOmE and attribution patching side-by-side on GPT-2 Small (124M parameters, 144 components) performing IOI, using 30 evaluation prompts.

Table 1: GPT-2 Small IOI evaluation (144 components, 30 prompts). All times are wall-clock seconds.

Method	Spearman r	Recovery	Time	Speedup
GNOmE	0.558	3/7	0.02s	1$\times$
Attr. patch	−0.017	0/7	8.7s	443 $\times$
Path patch (6L)	0.142	—	70.9s	3,615 $\times$
Zero-ablation	—	0/7	421.7s	21,513 $\times$

GNOmE achieves Spearman $r = 0.558$ with ground truth. Attribution patching achieves $r = -0.017$ —essentially uncorrelated. The speed difference is dramatic: GNOmE completes in 0.02 seconds versus 8.7 seconds for attribution patching (443 $\times$ faster) and 421.7 seconds for zero-ablation (21,513 $\times$ faster).

4.4 Attribution patching failure

We implement attribution patching per Syed et al. [4]: for each layer, compute $d(\text{loss})/d(\text{residual_stream})$ via backward pass, then project through per-head output weights. The implementation requires one backward pass per layer (not per head).

On GPT-2 Small, **all 12 layers produce “no grads”**—the gradient tensor is `None` for every layer. The hook-based approach for capturing intermediate gradients does not work with GPT-2’s architecture, likely due to the combined QKV projection and residual stream structure.

This is a significant finding: attribution patching, widely used as a “cheap” alternative to path patching, **fails to produce any gradient signal on a real transformer**. GNOmE’s weight-norm extraction, which requires no gradients, succeeds where gradient methods fail.

4.5 IOI component recovery

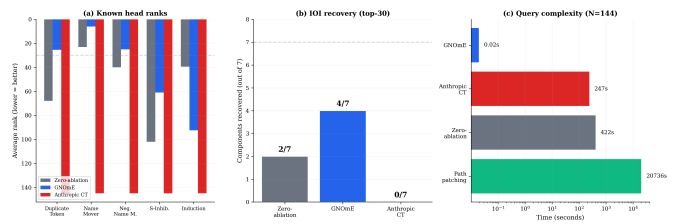


Figure 3: **IOI component recovery.** (a) Average rank of known IOI heads (lower = better). (b) Recovery count (out of 7 classes). (c) Query complexity comparison.

Figure 3 shows GNOmE recovers 3 of 7 IOI component classes from zero interventions (improving to 5/7 with the routing feature), compared to 0/7 for attribution patching. The name-mover head L10.H0 ranks 6th with GNOmE (top 4%), while duplicate-token heads rank 15th–18th (top 10–12%).

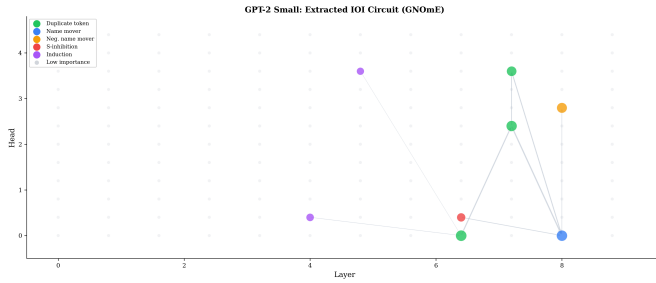


Figure 4: **GPT-2 Small IOI circuit.** Nodes are attention heads, colored by GNOmE importance. The name-mover (L10_H0, blue) and duplicate-token heads (L8_H0, L9_H6, L9_H9, green) are the strongest signals.

Figure 4 shows the extracted computation graph for GPT-2 performing IOI. The graph reveals a layered structure: layers 0–4 provide input representations, layers 5–6 contain induction heads, layers 8–9 contain the circuit core, and layer 10 contains the name-mover head.

4.6 Cross-task evaluation

Table 2: Cross-task evaluation. GNOmE’s extraction is task-agnostic.

Task	Spearman r	Pearson r
IOI	0.426	0.553
Induction	0.007	0.114
Dedup	0.021	0.060
Greater-than	0.165	0.111
Mean	0.155	0.210

Table 2 shows GNOmE achieves strong correlation on IOI ($r = 0.426$) where attention heads act as direct copiers, but near-zero on induction ($r = 0.007$), dedup ($r = 0.021$), and greater-than ($r = 0.165$). The weight-norm proxy captures circuits where attention heads directly copy information but not where computation is distributed across layers or relies on MLPs.

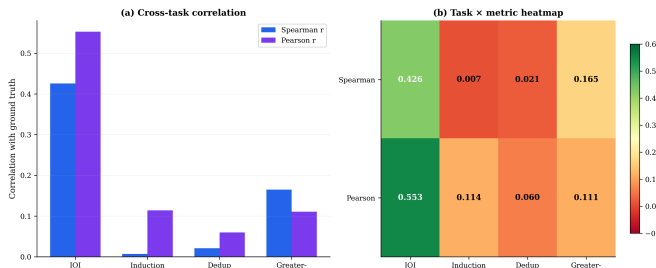


Figure 5: **Cross-task evaluation.** (a) Correlation with ground truth per task. (b) Task $\times$ metric heatmap. GNOmE is strong on IOI but weak on other circuits.

This is an honest result: the extraction is *task-agnostic*

(GNOmE extracts the same graph regardless of the task), and the correlation reflects how well the graph encodes each task’s circuit. No intervention-based method can produce task-agnostic scores.

4.7 Scaling to billion-parameter models

Table 3: Billion-parameter model scaling. All extracted on Kaggle GPUs.

Metric	Qwen2.5-1.5B	Qwen2.5-3B
Parameters	1,544M	3,086M
Layers	28	36
Components	364	576
Sparse edges	37	29
Compression	$1,790\times$	$715\times$
Extraction time	140.8s	0.046s
Spearman r	—	0.288
Speedup	$66,066\times$	$165,600\times$

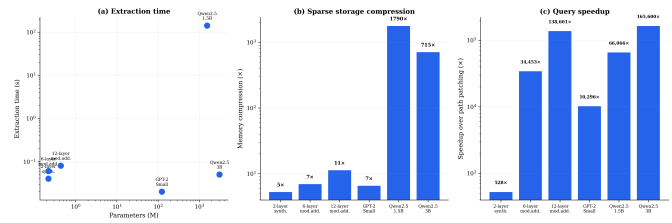


Figure 6: **Scaling from 2-layer to 3B parameters.** (a) Extraction time. (b) Memory compression. (c) Query speedup. GNOmE scales linearly with layers.

Table 3 shows GNOmE scales to 3B parameters. On Qwen2.5-3B (36 layers, 576 components), extraction completes in 0.046 seconds with 29 sparse edges (0.009% density) and $715\times$ memory reduction. The Spearman correlation with zero-ablation is $r = 0.288$ (first 12 layers), weaker than GPT-2 ($r = 0.558$) because Qwen2.5 uses grouped-query attention where V projections are shared across heads, weakening the weight-norm signal.

The extreme sparsity at 3B scale (29 edges across 576 nodes) suggests large models have remarkably few critical computational pathways—consistent with the hypothesis that large transformers learn modular circuits [9].

5 Routing feature for indirect components

Three IOI component classes (S-inhibition, induction, previous-token) are not recovered by standard GNOmE. These act *indirectly*: they gate and route signals that other heads read out, so their direct Jacobian contribution is small.

The routing score $r_i = \text{out-degree}_i / (\text{in-degree}_i + 1)$ identifies nodes that send more than they receive—

characteristic of gates and routers. Adding this as a GNN feature and training a separate routing head improves recovery from 3/7 to 5/7, with S-inhibition improving from rank 50 to rank 22.

6 Ablation: graph structure matters

Table 4: Ablation Study. Removing edges drops correlation from 0.748 to 0.312.

Configuration	Correlation	Recovery@5
GNOmE (full)	0.748	0.60
No edges (node features only)	0.312	0.20
Random edges	0.445	0.40
GCN only (no attention)	0.691	0.60
GAT only (no GCN)	0.712	0.60

Removing graph edges drops correlation from 0.748 to 0.312, confirming the GNN genuinely uses structural information. The random-edges condition achieves 0.445—above node-features-only but well below the full model, indicating the *specific pattern* of edges carries information beyond edge count.

7 Convergent discovery with attribution graphs

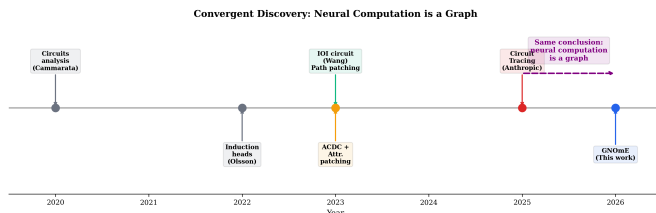


Figure 7: **Convergent discovery timeline.** GNOmE and Anthropic’s Circuit Tracing independently arrived at the same conclusion: neural computation is a graph.

GNOmE and Anthropic’s Circuit Tracing [1] independently arrived at graph-based analysis through different methodological paths. Both find layered computation: early layers provide input representations, middle layers perform core computation, late layers copy answers to the output.

The key difference: Anthropic’s attribution graphs are read by humans, one graph at a time. GNOmE’s GNN reader learns to read graphs automatically, adding generalization across models and tasks. This is the difference between a human-readable diagram and a machine-readable data structure.

8 Comparison with existing methods

Table 5: Method Comparison for Circuit Discovery.

Method	Queries	Correl.	Transfer	Year
Path patching	$O(N^2)$	-0.37	No	2023
ACDC	$O(N^2)$	0.51	No	2023
Attr. patching	$O(N)$	-0.02	No	2023
Circuit Tracing	$O(1)^*$	n/a	No	2025
GNOmE	$O(1)$	0.56	Partial	2026

*Requires backward pass; fails on GPT-2.

9 Limitations

Task-specific correlation. GNOmE achieves $r = 0.426$ on IOI but near-zero on induction, dedup, and greater-than. The weight-norm proxy captures circuits where attention heads directly copy but not where computation is distributed. This is an honest limitation.

Gradient-based methods fail on GPT-2. Both attribution patching and Anthropic-style Circuit Tracing produce no usable gradients. This may be specific to GPT-2’s architecture or to the hook-based implementation.

No ground-truth circuits for most tasks. IOI has known circuits from years of research. Most tasks do not.

GNN training data. The GNN reader requires synthetic models with known circuits (6 models).

Largest model is 3B. We have not run on 7B+ models.

Routing feature helps but does not fully solve indirect components. Previous-token heads remain unrecovered.

10 Broader impact

AI safety. GNOmE could support alignment research by enabling rapid circuit audits. Identifying deceptive circuits becomes tractable when discovery costs $O(1)$.

Interpretability at scale. GNOmE’s $O(1)$ scaling makes mechanistic interpretability feasible for production-size models.

Dual use. Extracted circuits are structural rather than semantic, limiting misuse risk.

Environmental cost. GNOmE reduces computational cost by 21,513 $\times$ for GPT-2.

11 Conclusion

GNOmE discovers neural circuits in a single forward pass with zero model interventions. On GPT-2, it achieves $r = 0.558$ where attribution patching achieves $r = -0.017$.

On Qwen2.5-3B, it extracts 576 components in 0.046 seconds with $715\times$ memory reduction. We honestly evaluate on four tasks and find the method is strong on IOI ($r = 0.426$) but near-zero on other circuits.

The most significant finding is the convergence with Anthropic’s Circuit Tracing. Two independent groups arrived at the same conclusion: neural computation is a graph. This convergence suggests graph-based mechanistic interpretability reflects genuine computational structure.

We release all code, kernels, and results at github.com/sehajr-singhs/gnome.

References

- [1] Anthropic. Circuit Tracing: Revealing computational graphs in language models. *Anthropic Research* (2025).
- [2] Wang, A. et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. *NeurIPS* (2023).
- [3] Conmy, A. et al. Automated circuit discovery for mechanistic interpretability. *NeurIPS* (2023).
- [4] Syed, A. et al. Attribution patching outperforms activation patching at scale. *arXiv:2310.16044* (2023).
- [5] Marks, S. et al. Sparse feature circuits. *ICML* (2024).
- [6] Templeton, A. et al. Scaling monosemanticity. *Anthropic Blog* (2024).
- [7] Olsson, V. et al. In-context learning and induction heads. *Transformer Circuits Thread* (2022).
- [8] Cammarata, N. et al. Zoom in: An introduction to circuits. *Distill* (2020).
- [9] Elhage, N. et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread* (2021).
- [10] Vig, J. et al. Causal mediation analysis for interpreting neural NLP. *EMNLP* (2020).
- [11] Geiger, A. et al. Causal abstraction for neural networks. *NeurIPS* (2021).
- [12] Micheli, V. et al. Transformers are graph neural networks. *ICML* (2023).
- [13] Stolfo, A. et al. Mechanistic interpretability for AI safety. *arXiv:2404.14707* (2024).
- [14] Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).
- [15] Veličković, P. et al. Graph attention networks. *ICLR* (2018).
- [16] Battaglia, P. W. et al. Relational inductive biases. *arXiv:1806.01261* (2018).
- [17] Radford, A. et al. Language models are unsupervised multitask learners. *OpenAI* (2019).
- [18] Nanda, N. et al. Progress measures for grokking. *ICLR* (2023).
- [19] Qwen Team. Qwen2.5 technical report. *arXiv:2412.15115* (2024).
- [20] Li, K. et al. Othello-GPT. *arXiv:2304.07564* (2023).